

ALGORITMOS CLÁSICOS Y ESTRUCTURAS DE DATOS

Proyecto Final: Sistema de Gestión de Rutas de Transporte

1. Introducción y Descripción General

El presente proyecto consiste en el diseño y desarrollo del "Sistema de Gestión de Rutas de Transporte", una aplicación orientada a gestionar y optimizar las rutas de transporte público dentro de un área urbana. El sistema provee una herramienta eficiente para calcular la mejor ruta entre dos puntos, tomando en consideración factores críticos como el tiempo de viaje, la distancia, los costos y el número de transbordos requeridos.

2. Decisiones de Diseño e Implementación Técnica

La implementación del sistema se dividió estratégicamente en el manejo de datos, lógica algorítmica y presentación visual:

- **Interfaz Gráfica:** Se desarrolló utilizando JavaFX para garantizar una experiencia de usuario interactiva y visual. Esta interfaz incluye formularios de entrada, paneles de información y una representación gráfica interactiva de la red de transporte.
- **Gestión de Datos:** Para asegurar la persistencia de la información entre sesiones, se implementó un almacenamiento en base de datos local (PostgreSQL mediante contenedores Docker), permitiendo guardar las paradas y las rutas de forma permanente.

3. Estructura de Datos

El núcleo de la aplicación modela la red de transporte utilizando la estructura de un grafo dirigido.

- **Nodos:** Representan las diferentes paradas de transporte disponibles en el sistema.
- **Aristas:** Representan las rutas que conectan dichas paradas, las cuales están ponderadas. Cada ruta cuenta con atributos específicos: tiempo estimado, distancia recorrida, costo del pasaje y cantidad de transbordos.

Para la representación en memoria, el grafo fue implementado utilizando listas de adyacencia. Esta decisión se justifica por la alta eficiencia que ofrecen las listas de adyacencia al trabajar con redes de rutas densas, optimizando el uso de memoria espacial frente a una matriz de adyacencia tradicional.

4. Análisis Asintótico y Complejidad Algorítmica

A continuación, se presenta el análisis de complejidad espacial y temporal aplicando las notaciones Big-O para la cota superior, Theta para la cota ajustada, y Omega para la cota inferior, en función del número de paradas (V) y el número de rutas (E).

4.1.

Operaciones sobre la estructura del grafo

- **Inserción de una parada:** $O(1)$ amortizado. Al utilizar un HashMap para almacenar los vértices del grafo, agregar un nuevo nodo es una operación de tiempo constante en promedio.
- **Inserción de una ruta:** $O(1)$. Consiste en acceder a la lista de adyacencia del nodo origen y añadir la nueva ruta al final de la lista vinculada.
- **Eliminación de una ruta:** $O(|V|)$. En el peor de los casos, se debe recorrer toda la lista de adyacencia de un nodo para encontrar y remover la arista específica. Su límite inferior es $\Omega(1)$ si es el primer elemento.
- **Consulta de vecinos de una parada:** $\Theta(1)$ para obtener la lista de adyacencia desde el HashMap, y $O(k)$ para iterarla, donde k es el grado de salida del vértice.
- **Verificación de existencia de una arista:** $O(|V|)$ en el peor escenario (un grafo fuertemente conectado donde se deba iterar toda la lista) y $\Omega(1)$ en el mejor.

4.2.

Algoritmos de Optimización Implementados

El sistema es capaz de calcular la ruta más rápida, la más corta o la de menos transbordos, permitiendo además calcular rutas alternativas. Las justificaciones de complejidad son las siguientes:

- **Algoritmos BFS y DFS:** Poseen una complejidad de $O(|V| + |E|)$. Esto se debe a que, en el peor de los casos, la búsqueda en anchura o profundidad debe explorar cada vértice y cada arista exactamente una vez utilizando las estructuras auxiliares (Pila o Cola).
- **Algoritmo de Dijkstra:** Tiene una complejidad temporal de $O((|V| + |E|) \log |V|)$. Esta cota superior está dictada por el uso interno de una cola de prioridad (PriorityQueue); el logaritmo refleja el costo computacional debido a la inserción y extracción en la cola de prioridad al evaluar y relajar cada ruta.
- **Algoritmo de Bellman-Ford:** Presenta una complejidad ajustada de $\Theta(|V| * |E|)$. Su implementación requiere iterar sobre todas las aristas (E) del grafo exactamente $V - 1$ veces para garantizar la relajación óptima, lo que justifica la multiplicación de ambos factores. Permite resolver problemas con aristas de peso negativo.
- **Algoritmo de Floyd-Warshall:** Su complejidad es exactamente $\Theta(|V|^3)$. Calcula la ruta más corta entre todas las paradas de forma general y se basa en tres bucles anidados que iteran sobre la cantidad total de vértices V para construir la matriz de distancias mínimas, lo que lo hace muy costoso computacionalmente para redes extremadamente grandes, pero con un tiempo de ejecución predecible.

5. Pruebas y Validación

Para garantizar el correcto funcionamiento del sistema y la exactitud de los algoritmos descritos, se procedió a probar los algoritmos con conjuntos de datos precargados que simulan una pequeña red de transporte urbano. Durante estas pruebas, se validó que el sistema cumple con el requerimiento funcional de mostrar rutas alternativas y proveer detalles precisos sobre el costo y tiempo de viaje.